

Rewindscope — Master Product Document

Rewindscope is a time-travel debugger for React Server Components (RSC), Server Actions, cache invalidation, streaming HTML, cookies/headers, and client transitions in modern Next.js apps. It records a full application session as a replayable execution trace and renders that run as a scrubbable timeline with diff views, source-code mapping, and event-level causality. The core problem it solves is that App Router applications are difficult to debug because state is split across server and client boundaries, rendering is streamed, cache invalidation changes behavior over time, and existing tools do not unify these layers into a single coherent debugging model.[web:2][web:98][page:1][page:2]

Executive Brief

One-line pitch

Rewindscope lets engineers rewind a Next.js session and inspect exactly which Server Action ran, which cache tags changed, what RSC payload streamed, what cookies/headers mutated, and how the client tree reacted — all in one timeline.[web:2][page:2]

Why now

Next.js has expanded its debugging surface through Server Actions, App Router rendering, cache tags, tag invalidation, and MCP runtime introspection, but official and platform tooling still centers on logs, traces, metrics, firewall rules, and agent-facing inspection rather than session-level replay debugging for humans.[web:2][web:98][page:1]

Problem and Concepts

What is App Router?

App Router is Next.js's modern routing and rendering system. It supports React Server Components, nested layouts, streaming rendering, and Server Actions, which means the UI is no longer purely client-side and may arrive in chunks over time.[page:1]

What are React Server Components?

React Server Components are components rendered on the server and sent to the client in a specialized payload. They reduce client bundle size and allow direct server-side data fetching, but they create a split execution model where developers must reason about both server and client rendering behavior.[page:1]

What is the RSC payload?

The RSC payload is the serialized server-rendered component data sent to the client so React can reconcile the UI. Security researchers and framework analysis commonly refer to this as the React “Flight” protocol, highlighting that it is a protocol-like serialized format rather than plain HTML.[web:1][web:106]

What are Server Actions?

Server Actions are server-side functions triggered from the UI, often by forms or mutations. They are important because they may trigger redirects, cache invalidation, cookie writes, or UI refreshes, and Vercel exposes Server Action names as first-class operational entities in firewall tooling.[web:2][web:5][page:1]

What are cache tags and invalidation?

Next.js lets developers associate data with tags, then invalidate those tags later. `updateTag` is used from Server Actions for immediate read-your-own-writes behavior, while `revalidateTag` may support stale-while-revalidate patterns depending on context, making cache behavior subtle and hard to debug visually.[page:2]

What is MCP in this context?

Next.js 16+ includes MCP support through `next-devtools-mcp`, exposing runtime information such as errors, logs, page metadata, routes, and `get_server_action_by_id`. This proves the ecosystem recognizes a richer debugging surface, but MCP remains an agent-oriented runtime access mechanism rather than a human-first replay debugger.[page:1][web:108]

Market Gap and Positioning

Existing products and gaps

Vercel Observability provides logs, performance views, outgoing request traces, and OTEL export, but it is not a time-travel debugger over the RSC and Server Action lifecycle.[web:98] Vercel Firewall can target specific Server Actions in Next.js 15.5+, which validates Server Actions as an operational surface, but firewall rules do not explain user-session causality.[web:2][web:5] Next.js MCP exposes runtime inspection tools such as `get_errors`, `get_logs`, and `get_server_action_by_id`, but this is still metadata and diagnostics rather than replay and diff-oriented UX.[page:1][web:108]

Rewindscope’s wedge

Rewindscope unifies six planes in one debugger: user actions, Server Actions, cache invalidation, RSC payloads, response mutations, and client tree mutations. That is the key differentiator versus observability, runtime metadata, or generic replay products.[web:98][page:1]

Core Product Blueprint

Main surfaces

- Recorder SDK
- Runtime collector
- Trace storage layer
- Inspector PWA
- Source mapper
- AI debug assistant

Feature set

- Record Server Action invocations and outcomes.
- Record RSC payload chunks and metadata.
- Record cache invalidation events such as `updateTag` and `revalidateTag`.[\[page:2\]](#)
- Record redirects, cookie writes, header mutations, and client transitions.
- Scrubbable timeline, tick-based diffs, tree snapshots, and payload explorer.
- `.rwd` replay bundle export.
- AI-generated summaries and bug reports.

High-Level Architecture

```
flowchart LR
    U[User Interaction] --> C[Next.js App]
    C --> SR[Server Runtime Instrumentation]
    C --> CR[Client Runtime Instrumentation]
    SR --> N[Trace Normalizer]
    CR --> N
    N --> LS[IndexeDB or OPFS]
    N --> EX[.rwd Exporter]
    LS --> I[Inspector PWA]
    EX --> I
    I --> AI[AI Debug Assistant]
    I --> SM[Source Mapper]
    SM --> MCP[Next.js MCP Adapter]
```

This design uses server-side and client-side capture, normalizes the events, persists them locally, and renders them in an inspector that can later be enriched by MCP metadata sources.

[\[page:1\]](#)

Low-Level Architecture

```
rewindscope/  
  apps/  
    inspector-pwa/  
    demo-playground/  
    docs-site/  
  packages/  
    recorder-core/  
    recorder-next/  
    recorder-client/  
    trace-schema/  
    trace-normalizer/  
    trace-storage-indexeddb/  
    trace-storage-opfs/  
    bundle-rwd/  
    source-mapper/  
    ai-insights/  
    mcp-adapter/  
    test-harness/
```

Event schema

```
export type TraceEvent = {  
  id: string  
  sessionId: string  
  ts: number  
  tick: number  
  phase:  
    | 'user-input'  
    | 'client-navigation'  
    | 'server-action:start'  
    | 'server-action:end'  
    | 'rsc:chunk'  
    | 'cache:update-tag'  
    | 'cache:revalidate-tag'  
    | 'headers:mutate'  
    | 'cookies:mutate'  
    | 'redirect'  
    | 'tree:diff'  
    | 'error'  
  route?: string  
  actionId?: string  
  actionName?: string  
  payloadRef?: string  
  sourceRef?: string  
  meta?: Record<string, unknown>  
}
```

ER Diagram

```
erDiagram
    SESSION ||--o{ TRACE_EVENT : contains
    SESSION ||--o{ SNAPSHOT : has
    SESSION ||--o{ BUNDLE_EXPORT : produces
    TRACE_EVENT ||--o{ ARTIFACT : references
    TRACE_EVENT ||--o{ SOURCE_MAP_REF : maps_to
    SESSION ||--o{ COMMENT : includes
    SESSION ||--o{ AI_INSIGHT : generates
```

Demo Examples

Demo 1 — blog post creator

A mutation runs as a Server Action, triggers `updateTag('posts')`, and redirects to the created post page. Rewindscope shows the exact action, invalidation, streamed payload, and refreshed view in one trace.[page:2]

Demo 2 — stale admin dashboard

A dashboard edit flow uses the wrong invalidation semantics, and the UI remains stale. Rewindscope reveals that the invalidation strategy does not match the expected read-your-own-writes behavior.[page:2]

Demo 3 — auth cookie mutation

A sign-in flow changes cookies and redirects through nested layouts. Rewindscope shows cookie mutation, redirect, and tree reconciliation to explain partial stale auth state.

Build Roadmap

Phase 0 — research and design

- Trace schema
- Synthetic fixtures
- Inspector information architecture
- Capture surface inventory

Phase 1 — recorder MVP

- Next.js instrumentation hook
- Client event capture
- Server Action capture
- Basic payload and cache event recording
- IndexedDB persistence

Phase 2 — inspector MVP

- Timeline UI
- Event list
- Tick details panel
- Simple diff view
- .rwd export/import

Phase 3 — demo quality

- Source mapping
- Payload explorer
- Demo apps
- Benchmarks

Phase 4 — growth

- VS Code extension
- MCP connector
- AI summarization
- Cloud sync alpha

Integrations and Distribution

Recommended order:

1. npm package + PWA inspector.
2. VS Code extension.
3. MCP connector.
4. Desktop app.
5. Browser extension overlay.

This order gives the lowest adoption friction and strongest daily workflow value first.[page:1]

Build in Public on X

Use a thesis-driven narrative:

- “App Router needs its Redux DevTools moment.”
- “I’m building a time-travel debugger for Server Actions and RSC.”
- “Why cache tags and streamed payloads make Next.js hard to debug.”

Publish weekly:

- technical concept thread,

- product demo clip,
- reverse-engineering insight,
- roadmap recap.

Future Scope

- Remix support.
- SvelteKit support.
- Team collaboration and cloud syncing.
- Browser video replay.
- Automatic source-code mapping for every event.
- Distributed tracing across external microservices.

Additional Requested Solutions

1. How to implement the RSC payload reconciliation logic for replay

Goal

The goal is not to execute React internals from scratch, but to create a deterministic replay model that lets the inspector explain how the UI changed over time. Rewindscope should reconstruct enough of the semantic tree state to show what streamed in, what changed, and which client-visible tree nodes were affected.

Important nuance

The RSC payload, often called the Flight protocol, is a serialized transport format. Because it is protocol-like and sensitive enough that malformed payloads have been linked to serious security issues, Rewindscope should treat it as structured data to parse, normalize, sanitize, and snapshot rather than blindly deserialize arbitrary payloads for execution.[web:1][web:106][web:109]

Recommended implementation strategy

1. Capture each RSC chunk as an ordered frame with session ID, request ID, route segment, chunk index, and timestamp.
2. Parse chunks into a normalized intermediate representation rather than trying to mirror raw transport bytes forever.
3. Build a replay graph of `treeNodeId` -> `versioned state` per tick.
4. Associate each payload chunk with the triggering event chain: user action, request, Server Action, cache invalidation, redirect, client navigation.
5. Materialize snapshots at stable reconciliation boundaries so the inspector can jump to ticks without replaying the full session from the beginning.

6. Store structural diffs between snapshots to reduce bundle size.

Suggested replay data model

```
type RscFrame = {
  sessionId: string
  requestId: string
  frameIndex: number
  ts: number
  routeSegment?: string
  rawKind: string
  normalizedOps: RscOp[]
}

type RscOp =
  | { type: 'node-create'; nodeId: string; parentId?: string; label: string }
  | { type: 'node-replace'; nodeId: string; nextLabel: string }
  | { type: 'prop-patch'; nodeId: string; key: string; nextValue: unknown }
  | { type: 'suspend'; nodeId: string }
  | { type: 'resolve'; nodeId: string }
  | { type: 'remove'; nodeId: string }
```

Reconciliation algorithm

```
For each session:
  group frames by request/action lineage
  sort by timestamp and frame index
  initialize empty replay tree
  for each frame:
    for each normalized op:
      apply op to replay tree
    if frame creates a stable UI boundary:
      persist checkpoint snapshot
      emit tree:diff event
```

Stable boundary heuristics

Create a checkpoint when:

- a request completes,
- a redirect occurs,
- a suspense boundary resolves,
- a Server Action ends,
- a new route segment becomes visible.

Why this approach is right

This keeps the inspector fast, deterministic, and explainable. It avoids building a fragile shadow React runtime while still giving users the visibility they need.

2. Benchmarking Rewindscope against OpenTelemetry tracing overhead

Why benchmark this way

OpenTelemetry explicitly notes that instrumentation is not free and that the best way to measure overhead is in the context of a real application under load.[web:104] The OTel performance benchmark spec also recommends measuring throughput, CPU, and memory, using warm-up phases, multiple repetitions, and at least 15-second test windows with clear span/event configurations.[web:107] That makes OTel a useful baseline for “acceptable observability overhead” rather than a direct product competitor.

Benchmark objective

Measure Rewindscope overhead relative to:

- no instrumentation,
- lightweight custom logging,
- OpenTelemetry tracing baseline,
- Rewindscope capture modes: minimal, normal, verbose.

Metrics to collect

- request throughput
- median, p95, p99 latency
- CPU usage
- memory usage
- trace bundle size per session
- client FPS in inspector replay
- time to first interactive in demo app with recorder enabled

Test matrix

Scenario	Instrumentation	Load	Expected lesson
A	none	100 RPS	baseline
B	OpenTelemetry only	100 RPS	reference instrumentation overhead [web:107]
C	Rewindscope minimal	100 RPS	local capture impact
D	Rewindscope normal	100 RPS	realistic dev usage

Scenario	Instrumentation	Load	Expected lesson
E	Rewindscope verbose	100 RPS	worst-case ceiling
F	OTel + Rewindscope	100 RPS	coexistence cost

Methodology

- Use the same Next.js demo app for all scenarios.
- Warm up before measurement, as OTel guidance recommends.[web:107]
- Run at least 10 iterations per scenario, again matching OTel benchmark guidance.[web:107]
- Keep exporters out-of-process or dropped to avoid skewing application CPU, as the OTel benchmark spec recommends for OTLP benchmarking.[web:107]
- Record both server-side and client-side overhead.

Success target

A strong target is: Rewindscope in normal mode should remain within a bounded development-only overhead envelope, such as low single-digit to low double-digit percentage latency impact on demo flows, with a clear documented path to minimal mode for larger apps. This document should not invent exact numbers before measuring them; instead it defines the benchmark discipline inspired by OTel's published guidance.[web:104][web:107]

3. Strategies for integrating MCP connectors into the debugger workflow

Why MCP matters

Next.js MCP gives access to errors, logs, page metadata, project metadata, and `get_server_action_by_id`, which is directly relevant to mapping runtime events back to source.[page:1][web:108] This makes MCP an enrichment layer for Rewindscope rather than the primary recorder.

Integration strategy

Use MCP in three layers:

Layer 1 — metadata enrichment

- Resolve `actionId` -> file path + symbol via `get_server_action_by_id`. [page:1][web:108]
- Pull route/page metadata for context panes.
- Pull current logs and errors to correlate with ticks.

Layer 2 — AI copilot workflow

- Expose a Rewindscope MCP server so coding agents can query session traces.
- Let agents answer questions such as “Which Server Action triggered this stale state?” or “Show me all traces where route X had cookie mutation plus redirect.”

Layer 3 — developer workflow bridge

- From inspector tick -> open in VS Code.
- From VS Code -> request latest trace for current route.
- From trace -> open related Next.js MCP metadata in side panel.

Architecture pattern

```
flowchart LR
    T[Rewindscope Trace] --> E[Enrichment Engine]
    M[Next.js MCP] --> E
    E --> I[Inspector Context Pane]
    E --> A[AI Assistant]
    E --> V[VS Code Extension]
```

Important boundary

MCP should never be the sole source of truth for replay state. If the replay depends on a live dev server metadata call, it will become brittle. Record first, enrich second.

4. Designing the PWA inspector UI for complex state transitions

UI principle

Complex state transitions should be visualized as coordinated panes around a shared scrubber. The user should never have to mentally correlate multiple timestamps by hand.

Recommended layout

- Top: global scrub timeline.
- Left sidebar: session/event index and filters.
- Center main panel: selected view (causality graph, tree diff, payload explorer).
- Right context panel: source mapping, tags, cookies, headers, AI summary.
- Bottom optional tray: raw event JSON and console correlation.

Core interaction model

At any tick, all panes update together:

- selected action,
- cache state changes,
- RSC frames,
- client tree diff,
- source references,
- route metadata.

UI states

The inspector should support these modes:

- **timeline mode** for linear session understanding,
- **diff mode** for before/after comparison,
- **causality mode** for event graph reasoning,
- **payload mode** for protocol-level exploration,
- **presentation mode** for demos and bug reports.

Visual design guidance

The landing-page and inspector should feel documentation-grade and engineering-first, with restrained visual language similar to a shadcn/Vercel-style product aesthetic. Avoid noisy gradients and instead rely on typography, subtle borders, segmented panes, and precise motion.

Performance strategy

- Virtualize long event lists.
- Lazy-load heavy payload details.
- Use checkpoint snapshots so scrubbing does not recompute entire sessions.
- Use canvas or optimized DOM rendering for large timelines.

5. Optimizing Server Action cache invalidation monitoring for Next.js

Problem

The hard part is not just recording that invalidation happened; it is explaining whether the invalidation semantics matched developer intent. The distinction between `updateTag` and `revalidateTag` is especially important because they imply different freshness behavior.[page:2]

Monitoring strategy

1. Capture all invalidation events with source location, action ID, tag, route, and timestamp.
2. Correlate each invalidation event to the triggering Server Action and subsequent RSC request.
3. Compute a semantic outcome summary:
 - immediate refresh observed,
 - stale response observed first,
 - no visible tree update observed,
 - invalidation issued without downstream fetch usage.
4. Surface this in a dedicated “Cache Semantics” panel.

Example classification

```
type CacheOutcome =  
  | 'immediate-freshness'  
  | 'stale-then-refresh'  
  | 'no-visible-effect'  
  | 'orphaned-invalidation'
```

Why this matters

This turns cache monitoring into a debugging product. Instead of showing a raw log line saying “tag invalidated,” Rewindscope tells the engineer whether the user actually saw fresh data and how long it took.

Recommended heuristics

- If `updateTag` occurs in a Server Action and the next relevant RSC payload changes the dependent tree immediately, classify as immediate freshness.[page:2]
- If invalidation occurs but the next rendered tree still reflects old data before a later update, classify as stale-then-refresh.[page:2]
- If invalidation occurs and no tracked tree node changes for the affected route, classify as no-visible-effect.
- If tag invalidation appears but no tracked fetch or dependent subtree references the tag, classify as orphaned invalidation.

Testing, QA, and CI/CD Addendum

Unit testing focus

- RSC frame parser.
- reconciliation engine operations.
- invalidation outcome classifier.
- MCP enrichment adapter.
- bundle import/export compatibility.

Automated integration tests

- Use Next.js demo apps with real Server Actions.
- Capture traces, import them into the inspector, and assert on expected tick semantics.
- Include compatibility tests across supported Next.js patch versions because framework internals may drift.[web:1]

Regression testing

- Golden trace bundles for all demos.
- Visual regression snapshots for the inspector.
- Performance regression thresholds for capture overhead.

CI/CD workflow

- lint
- typecheck
- unit tests
- integration tests
- Playwright E2E
- benchmark smoke tests
- docs build
- release artifacts

Claude Code Prompts for Visual Diagrams and Landing Page

Docs-style landing page prompt

"Build a documentation-style landing page for Rewindscope using a shadcn-inspired design system. Include a hero with clear technical value proposition, left-nav docs layout, an interactive scrub timeline demo, comparison sections, architecture diagrams rendered from Mermaid, code snippet cards, and restrained Vercel-like styling. Prioritize readability, white space, precise borders, and a trustworthy engineering aesthetic."

Interactive debugger mock prompt

"Create an interactive web demo of a time-travel debugger for Next.js App Router. A horizontal scrubber controls synchronized panes for Server Actions, cache tag invalidations, RSC payload frames, and client tree diffs. Use synthetic data for 12 ticks. Animate transitions lightly and keep performance high."

Concept explainer animation prompt

"Create a lightweight explainer animation for a documentation page showing: user clicks save, a Server Action runs, updateTag invalidates cache, new RSC payload frames stream in, and the client tree updates. The animation should clearly teach the concept in under 10 seconds with labels and subtle arrows."

Closing

Rewindscope is strongest when framed as the missing replay debugger for the App Router era. Vercel and Next.js have already signaled that Server Actions, runtime metadata, and richer introspection matter through firewall support, observability products, and MCP support, but those surfaces still do not provide a unified human-first replay of causality across actions, cache, RSC, and client reconciliation.[web:2][web:98][page:1][web:108]